

Les dictionnaires en Python

Dictionnaire ?

- Un dictionnaire est un objet **non ordonné** qui contient plusieurs objets associés chacun à une **clé** de type **immuable** (souvent entiers et/ou chaînes de caractères)
- Chacun des objets contenus dans un dictionnaire peut être obtenu à l'aide de sa **clé**
- Une clé ne peut apparaître qu'**en un seul exemplaire (clé unique)**
- L'accès à un objet **est rapide** (temps constant en moyenne)

Exemple.

```
>> dico = {} # Def d'un dic vide
>> dico['chair'] = 'chaise'
>> dico['table'] = 'table'
>> Print(dico)
{'table': 'table', 'chair': 'chaise'}
>> print dico['chair']
chaise
>> print (dico['chaise'])
KeyError: 'chaise'
```

Opérations sur les dictionnaires

- 🔹 Création : dictionnaire vide
`{ }`
- 🔹 Ajout/modification d'un élément
`dico[cle]=valeur`
- 🔹 Destruction d'un élément
`del dico[cle]`
- 🔹 longueur (nombre d'entrées)
`len(dico)`

Méthodes des dictionnaires



Liste des clés

```
dico.keys()
```



Liste des valeurs

```
dico.values()
```



Existence d'une clé

```
dico.has_key(cle)
```



Nettoyage

```
dico.clear()
```



Copie superficielle

```
dico2 = dico.copy()
```

Méthodes élaborées des dictionnaires



Obtention de valeur sure

```
dico.get(cle, valDef)
```

Fournit la valeur associée à `cle` ou bien `valDef` en cas d'absence



Liste des paires

```
dico.items()
```

Fournit la liste des tuples `(cle, valeur)`

Les méthodes d'un dictionnaire

Method	Description
<u>clear</u> ()	Removes all the elements from the dictionary
<u>copy</u> ()	Returns a copy of the dictionary
<u>fromkeys</u> ()	Returns a dictionary with the specified keys and value
<u>get</u> ()	Returns the value of the specified key
<u>items</u> ()	Returns a list containing a tuple for each key value pair
<u>keys</u> ()	Returns a list containing the dictionary's keys
<u>pop</u> ()	Removes the element with the specified key
<u>popitem</u> ()	Removes the last inserted key-value pair
<u>setdefault</u> ()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<u>update</u> ()	Updates the dictionary with the specified key-value pairs
<u>values</u> ()	Returns a list of all the values in the dictionary

Exemple1 – Codage aléatoire des lettres

Écrire un programme qui fabrique un dictionnaire de codage (`code`), établissant une correspondance entre les lettres de l'alphabet. On ne s'intéresse qu'aux lettres minuscules. On suppose disposer d'une liste nommée `lettres` contenant tous les caractères de l'alphabet.

```
from random import randint
code={}
lettres='abcdefghijklmnopqrstuvwxyz'
lettres2=list(lettres)
for L in list(lettres) :
    j = randint(0,len(lettres2)-1)
    code[L]=lettres2[j]
    del lettres2[j]
print(code, 'len= ', len(code))
```

Outputs:

```
{ 'a': 'd', 'b': 'n', 'c': 'p', 'd': 'l', 'e': 'w', 'f': 'q', 'g': 'h',
  'h': 'j', 'i': 'i', 'j': 'e', 'k': 'g', 'l': 'c', 'm': 'b', 'n': 'o',
  'o': 't', 'p': 's', 'q': 'a', 'r': 'r', 's': 'v', 't': 'm', 'u': 'y',
  'v': 'z', 'w': 'k', 'x': 'f', 'y': 'x', 'z': 'u'}, len= 26
```

Exemple2- Affichage

On suppose disposer d'une variable dictionnaire **dico**. Ecrire un programme qui affiche à raison d'une paire par ligne toutes les clés et toutes les valeurs du dictionnaire. Afficher également tous les point fixes (c'est à dire les éléments du dictionnaire dont la valeur et la clé sont égales).

```
codage={ 'a' : 'x' , 'b' = 'q' , 's' : 's' , 'y' = 'd' , 'd' : 'd' }  
for cle in codage.keys() :  
    print ( cle, codage[cle])  
  
for cle in codage.keys() :  
    if cle == codage[cle] :  
        print ( cle, 'est un point fixe')
```

Exemple 3 : Les clés de type tuple

Il n'est pas toujours souhaitable d'utiliser des chaînes comme clé de dictionnaires

Par exemple on peut utiliser un dictionnaire pour stocker un tableau partiel (une carte)

	1	2	3	4	5	
1	T					X : Pieu
2				X		T : Tresor
3		X				
4			X			
5	X					

Exemple 3 : Les clés de type tuple

```
tresor = {}  
tresor[(1,1)] = 'trésor'  
pieges = [(2,4), (3,2), (4,3), (5,1)]  
  
for piege in pieges :  
    tresor[piege] = 'piège'  
  
tentative=tuple(int(x) for x in input("Donnez  
les coordonnées").split(','))  
resultat = tresor.get(tentative, "rien")  
print ("Vous avez trouvé",resultat)
```

Exemple 4 : Histogramme

On dispose d'une chaîne de caractères `ch`. Ecrire un programme qui fabrique un dictionnaire mettant en correspondance chaque caractère avec le nombre de ses occurrences dans `ch`

```
ch = "les chaussettes de l'archi-duchesse \
sont-elles sèches archi-sèches"
histo={ }
```

```
for car in ch:
    histo[car]=histo.get(car,0)+1
```

Imbrication de données

- **Imbriquer** des données
=> Insertion d'une structure de données comme élément d'une autre
- **Contraintes** selon la structure principale
=> Éléments d'un ensemble et clés d'un dictionnaire **non modifiables**
- Représentation de **données complexes**
=> Bien organiser et définir la structure de données

Parcours d'un dictionnaire

- Accès aux clés avec la fonction **keys** et aux paires avec **items**
Renvoient des séquences que l'on peut convertir en liste

```
fruits={'lemon':5.75,  
       'pear':21.50,  
       'apple':14.25  
}  
  
print('Fruits Dict : ',fruits)  
print('Fruits List : ',list(fruits.keys()))  
print('Prices List : ', list(fruits.values()))  
print('Pairs List : ',list(fruits.items()))  
#parcours par fruit  
for fruit in fruits:  
    print(fruit,fruits[fruit],sep=':',end=' ')  
#ou bien  
print()  
for fruit in fruits.keys():  
    print(fruit,fruits[fruit],sep=':',end=' ')  
#parcours par pair (by item)  
print()  
for fruit,price in fruits.items():  
    print(fruit,price,sep=':',end=' ')
```

Outputs:

```
Fruits Dict : {'lemon': 5.75, 'pear':  
21.5, 'apple': 14.25}  
Fruits List : ['lemon', 'pear', 'apple']  
Prices List : [5.75, 21.5, 14.25]  
Pairs List : [('lemon', 5.75), ('pear',  
21.5), ('apple', 14.25)]  
lemon:5.75, pear:21.5, apple:14.25,  
lemon:5.75, pear:21.5, apple:14.25,  
lemon:5.75, pear:21.5, apple:14.25,
```

Exemple 5 : Somme des durées d'une playlist

- **Liste de musiques** dont il faut calculer la durée totale
- Chaque musique est stockée avec un **dictionnaire**
Passer en revue chaque dictionnaire et extraire la durée

```
chanson1={'titre':'Alkamar Al ahmar','chanteur':'Belkhaïat',  
         'compositeur':'Abdessalam amer','duree':10, 'score':8 }  
chanson4={'titre':'Bouhali', 'chanteur':'Belkhaïat',  
         'compositeur':'Rachdi', 'duree':4, 'score':5 }  
chanson2={'titre':'Marsoul Lhob', 'chanteur':'Doukali',  
         'compositeur':'Rachdi', 'duree':5, 'score':7 }  
chanson3={'titre':'Yaka Jerhi', 'chanteur':'Samih',  
         'compositeur':'Rachdi', 'duree':5, 'score':8 }  
ma_play_list=[chanson1,chanson2,chanson3,chanson4]  
  
def get_duree_totale(play_list):  
    total=0  
    for chanson in play_list:  
        total+=chanson['duree']  
    return total  
  
print('Durée totale : ', get_duree_totale(ma_play_list))
```

Outputs:

Durée totale : 24

Exemple 6 : Liste des titres interprétés par un artiste

Parcours du dictionnaire, à la recherche de l'artiste

Construction d'une **liste des titres**

```
def get_titres_artiste(ply_list,artiste):  
    lst_titres=[]  
    for chanson in ply_list:  
        if(chanson['chanteur']==artiste):  
            lst_titres.append(chanson['titre'])  
    return lst_titres  
print("titres de Belkaiat : ",get_titres_artiste(ma_play_list,'Belkhaiat'))
```

Outputs:

```
titres de Belkaiat :  ['Alkamar Al ahmar',  
                      'Lbouhali']
```

Exercices

Exercice1

Définir les listes suivantes **par compréhension** :

1. les carrés de tous les entiers compris entre 1 et 100 (inclus) ;
2. les couples (x, y) d'entiers positifs tels que $x + y = 100$;
3. les couples d'entiers de la forme (x, y) tels que $x \leq y$ et les deux entiers x et y sont tous deux inférieurs à 10 (indication : il est plus facile de les ranger par y croissants) ;
4. les mêmes qu'à la question précédente, mais rangés par x croissants ;
5. les entiers compris entre 1 et 1000 qui ne sont multiples ni de 3 ni de 5.

Exercice2

Objectif pour tout cet exercice : la réponse à chaque question tient en 1 ligne. On fera bon usage de la fonction `sum`.

1. Calculer $\sum_{1 \leq i \leq j \leq 100} i^j$.

2. On représente une matrice comme la liste de ses lignes.

Écrire une fonction `sommes_lignes(M)` qui renvoie $[S_1, \dots, S_n]$ où S_i est la somme de la i -ème ligne de M .

3. Écrire une fonction `Id(n)` qui renvoie la matrice identité $I_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{pmatrix}$.

Exercices

Exercice3 - Mémoïzation à l'aide de dictionnaires en Python

La *mémoïzation* consiste à stocker en mémoire une valeur pour ne pas avoir à la recalculer si elle s'avérait utile à nouveau par la suite. Nous allons en voir quelques applications.

Pour tout entier $p \in \mathbb{N}^*$, on définit la suite de Syracuse associée par

$$u_0 = p \text{ et } \forall n \in \mathbb{N}, u_{n+1} = \begin{cases} u_n / 2 & \text{si } u_n \text{ pair} \\ 3u_n + 1 & \text{sinon} \end{cases}$$

On admet que pour toute valeur raisonnable de p^1 , la suite finit par boucler sur 1, 4, 2. On note $\text{vol}(p)$ le plus petit n tel que $u_n = 1$, lorsque $u_0 = p$.

1. Quelle est la plus grande valeur de $\text{vol}(p)$ pour $1 < p < 1000$? Et jusqu'à $p = 10^7$?
2. Combien d'entiers $p \leq 10^6$ vérifient $\text{vol}(p) \leq 30$?
3. Optionnel : combien d'entiers p (sans limite de valeur !) vérifient $\text{vol}(p) \leq 50$?